

심화 실습 평가2(리눅스 프로그래밍)

1. 프로젝트 개요

- TCP 통신 프로토콜을 활용해 원격으로 여러 모듈들을 제어하는 프로그램을 제작해본다
- 서버는 라즈베리파이4, 클라이언트는 가상머신 우분투 리눅스에서 실행
- 우분투에서 컴파일해서 라즈베리파이에서도 실행할 수 있도록 크로스컴파일 환경 구축을 목표

1) 개발 환경

- 하드웨어
 - Raspberry Pi 4
 - 모듈 : LED, buzzer, 조도센서, 7세그먼트
- 소프트웨어
 - OS : Raspbian ___, Ubuntu ___
 - 언어 : C
 - 라이브러리 : wiringPi, pthread, softPwm, softTone
 - 빌드 도구 : make
 - 기타 도구 : VSCode, MobaXterm, ssh, git

2) 시스템 구성도

- 클라이언트 ↔ 서버 ↔ 각 장치 모듈

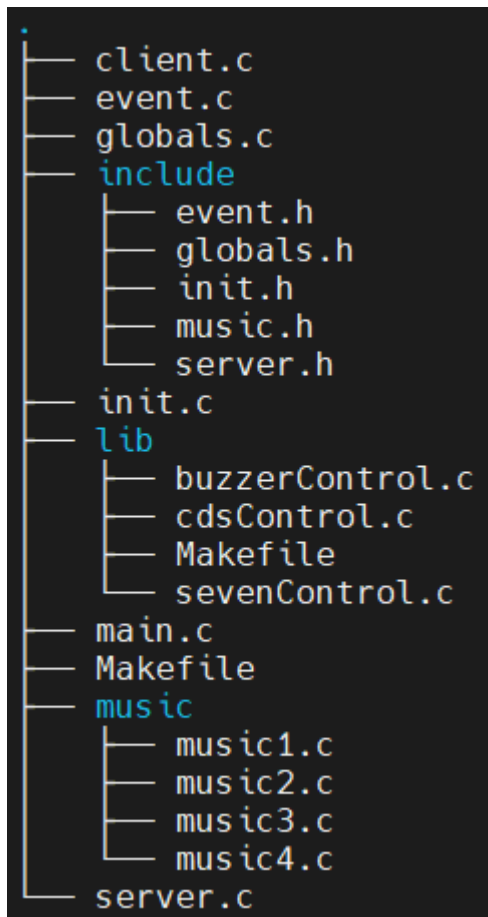
2. 개발 일정

기간(날짜)	작업 내용
12/23	요구사항 분석 및 모듈 기능 테스트
12/24	장치 기능, 서버 통신 등 전체 프로토타입 환경 구축
12/27	장치 기능 멀티 스레드로 리팩토링 및 Make 자동화
12/28	스레드 함수 동적 라이브러리화, 서버 데몬 적용

기간(날짜)	작업 내용
12/29	최종 테스트 및 버그 수정, 문서 작업

3. 시스템 구조 및 설계

3-1. 전체 구조 개요



- 멀티 프로세스 / 멀티 스레드 방식
- 주요 구성 요소
 - **main** : 서버 데몬화 및 구동
 - **server** : epoll 구조를 택해 client의 입력을 1초마다 받음
 - **client** : 사용자 명령 입력 및 서버로부터 결과 메시지 받아서 출력
 - 장치 모듈 : LED, buzzer, 조도센서, 7세그먼트

3-2. 스레드 구조

- 생성하는 스레드 목록
 - buzzer 스레드
 - 조도센서 스레드
 - 7세그먼트 스레드
- 각 스레드 간 공유 자원 & 동기화(mutex)
 - 전역 상태 변수 → 스레드간 통신을 통해 관리
 - 사용한 동기화 방법 : `pthread_mutex_t` , `pthread_cond_t`

3-3. 동적 라이브러리 구조

라이브러리 이름	파일명(.so)	함수 목록	기능
부저 제어	<code>libbuzzer.so</code>	<code>buzzer_thread()</code>	음악 재생
조도센서 제어	<code>libcds.so</code>	<code>cds_thread()</code>	조도값 측정 및 임계값 비교
7세그 제어	<code>libseven.so</code>	<code>seven_thread()</code>	숫자 카운트다운 표시

3-4. music 디렉토리

- buzzer 에서 재생할 음악들을 music 에 미리 정리해놓음
- 이후 MUSIC 구조체에 음악들을 담아서 재생

4. 장치 제어 세부 구현 내용

- 명령어 조합을 미리 정해놓고 그에 대응하는 handler 함수들을 구현
- 모든 명령어는 대소문자 구분 없이 가능
- LED와 music은 클라이언트 입력 받았을 때 handler 함수에서 제어
- 나머지 모듈은 handler 함수에서 상태 전역 변수들을 갱신해 각 thread 함수에서 제어

```
typedef struct cmd_type {
    char* cmd;
    void (*cmd_func)(int, char*);
} CMD_TYPE;

CMD_TYPE client_cmds[] = {
```

```
{ "help", handle_help },
{ "LED", handle_led },
{ "buzzer", handle_buzzer },
{ "cds", handle_cds },
{ "seg", handle_seven },
{ "music", handle_music },
{ NULL, NULL },
};
```

4-1. LED 제어

- 명령어: LED ON/OFF/MAX/MID/MIN
- 기본적으로 softPwmwrite로 0~100 범위 설정해서 on/off 및 밝기 조절
- ON/OFF 여부와 상관없이 MAX/MID/MIN 입력하면 미리 define된 밝기로 지정

4-2. buzzer / music

- 명령어: buzzer ON/OFF
- 음악 재생 제어
- 명령어: music 1~4
- buzzer에서 재생하는 음악을 미리 지정된 음악 중에 선택해서 재생하는 기능

4-3. 조도센서(CDS)

- 명령어: cds
- 조도센서가 감지한 조도값을 서버로부터 받음
- 정해진 boundary 값을 넘을 정도로 어두우면 led가 켜지고 밝으면 led가 꺼짐

4-4. 7세그먼트

- 명령어: seg 0~9
- 지정된 숫자만큼 7세그먼트에 숫자가 표시되고 1초마다 1씩 감소되며 0초가 되면 부저에서 종료음 울림
- busy_wait 상태를 개선하기 위해 pthread_cond_wait 사용

5. 예외 처리 및 안정성

- INT 신호 처리
 - 클라이언트 : SIGINT 시 소켓 정리 후 종료
 - 서버(데몬) : `kill` 수신 시 스레드 종료 플래그 설정, 자원 해제 후 종료
 - 스레드 종료 방법
 - `stop_flag` 등 전역 플래그 사용
 - 각 스레드 loop에서 플래그 확인 후 `pthread_exit`
 - 오류 처리
 - 소켓/epoll 오류, I2C 오류, 라이브러리 로딩 실패 시 처리 방법
 - 자원 해제
 - 소켓 `close`, `dlclose`, `pthread_join`, GPIO 원복 등
-

6. 문제점 및 보완 사항

- 사용자 입력을 받았을 때 thread를 생성하는 것이 아니라 서버 시작할때부터 루프문을 돌리며 조건을 검사하기 때문에 `busy_wait` 상태가 생겨 cpu 자원을 다소 잡아먹을 수 있음
 - `detach` 기능을 이용해 입력을 받았을 때 스레드를 생성해 자동으로 종료까지 하게 개선 가능
- 멀티스레드 환경에서 많은 전역변수를 사용하여 구현이 복잡해질 시 `race condition` 가능성
 - 상태 플래그들을 정확한 목적에 맞게 사용하며 기능 구현은 벗어나지 않게 최대한 변수 최소화 시키기
- 다양한 음악을 미리 작성했으나 buzzer가 어느정도 높은 음은 제대로 구현하지 못함
 - 표현할 수 있는 주파수 대역이 더 넓은 buzzer 장치로 교체
- 서버를 `daemon(1,0)` 로 데몬 프로세스로 만들었기 때문에 로그 파일을 남기지 않는 문제점이 있음
 - `syslog`에서 확인할 수 있도록 직접 데몬프로세스로 만드는 함수를 구현